Sanad — Project Scaffold

What was set up, why it is built this way, and the project's CLAUDE.md

What I did

```
Sanad/
├── .gitignore          ← one ignore file for the whole repo
├── package.json        ← root: "npm run dev" runs both apps
├── CLAUDE.md           ← gitignored
├── backend/
│   ├── .env.example
│   ├── tsconfig.json
│   ├── package.json    ← added dev / build / start / typecheck scripts
│   └── src/
│       ├── app.ts            ← builds the Express app (middleware, routes, errors)
│       ├── server.ts         ← HTTP server + Socket.IO + DB connect + listen
│       ├── config/env.ts     ← validates env vars at startup
│       ├── config/db.ts      ← Mongoose connect / disconnect
│       ├── middleware/errorHandler.ts
│       ├── utils/AppError.ts
│       ├── modules/          ← empty: auth, circles, stories... go here
│       └── sockets/          ← empty: chat / presence handlers go here
└── frontend/
    ├── .env.example
    ├── app/            ← routes (page.tsx = landing placeholder, layout.tsx = "Sanad"
title)
    ├── components/ui/      ← small reusable pieces (Button, Avatar)
    ├── components/layout/  ← navbar, sidebar, page shells
    ├── lib/api/        ← fetch wrappers (talk to Express)
    ├── hooks/          ← loading / error / data logic
    ├── context/        ← auth context
    └── types/          ← shared TypeScript types
```

The empty folders each hold a `.gitkeep`, because git doesn't track empty folders. Delete it once a real file lands there. I also replaced the Next.js boilerplate `page.tsx` with a placeholder and set the site title and description.

Why it's built this way

1 `app.ts` and `server.ts` are separate

What: `app.ts` only builds the Express `app`. `server.ts` runs it: it creates the HTTP server, attaches Socket.IO, connects the DB and listens.

Why not one file: tests import `app` and call it directly, with no open port, no live socket server and no DB connection started on import. It's the same split as Diraya.

Concept: separation of construction from execution.

2 Express and Socket.IO share one HTTP server

What: `http.createServer(app)`, then `new Server(httpServer)`.

Why: one port, one deploy, one CORS origin. And the WebSocket handshake starts as a normal HTTP request, so it carries the same auth cookie as your API calls. That's how you'll authenticate sockets later.

> **Interview line:** "Socket.IO upgrades an HTTP connection, so I attached it to the same server and reused the cookie-based auth for the handshake."

3 Env vars are validated with zod at startup

What: `env.ts` parses `process.env` once. If something is missing, the `app` exits with a clear message.

Why not read `process.env.X` everywhere: a missing `MONGODB_URI` would otherwise surface as a confusing crash much later, possibly in production. The rest of the code imports a typed `env` object.

Concept: fail fast.

4 `cors({ credentials: true })` with a single exact origin

Why: the frontend (port 3000) and API (port 5000) are different origins. Browsers only send cookies cross-origin when `credentials` is on, and that requires a specific origin rather than `*`.

5 `express.json({ limit: "100kb" })`

Why: caps request body size, so nobody can send a huge payload to exhaust memory. Story posts are text and fit easily.

6 Graceful shutdown (`SIGINT` / `SIGTERM`)

Why: hosting platforms send `SIGTERM` when they redeploy. Closing sockets and the DB connection cleanly avoids dropped connections and half-finished work.

7 One root `.gitignore`

Why: one file covers both apps. `.env.*` is ignored but `!.env.example` is kept, so secrets never get committed while the templates do. `frontend\.gitignore` from Next.js still exists; the two overlap harmlessly.

Notes (not applied)

- **Tests:** nothing here needs tests yet, since there are no business rules. The test setup (node:test + tsx, separate `sanad_test` database) comes with the first real module, auth. I'll walk you through the plan before writing it.
- `AGENTS.md` in the frontend was written by Next.js 16 and says it re-adds itself on `next dev`. Leave it and commit it.
- **No src/ in the frontend:** you chose not to use one in create-next-app. That's fine and officially supported. Config files and `app` folders just sit side by side. Switching later only means moving folders and changing one path in `tsconfig.json`.
- Next.js 16 renamed `middleware.ts` to `proxy.ts`. Your auth-based routing will go there. I noted it in `CLAUDE.md` so neither of us uses the old name.
- Unused Next.js SVGs are still in `public/`. Delete them whenever you like.

Suggested commit

`chore: scaffold Sanad monorepo — Next.js frontend, Express + Socket.IO + MongoDB backend skeleton`

Your CLAUDE.md, in full

The working preferences (everything before “Project context”) are copied word for word from Diraya. The project context is new, written for Sanad.

```
# Working Preferences

## Never change anything without explicit instruction
- Do NOT apply, change, delete, or add anything (code, files, config) unless I clearly tell you to.
- If my message doesn't explicitly ask for a change, don't make one. Answer, explain, or suggest instead.
- When I do ask for a change, change **only** what I asked for. No extra refactors, cleanups, renames, or "while I'm here" edits.

## Do share observations freely
- Feel free to point out anything you notice that's worth changing, updating, removing, or that needs attention (bugs, inconsistencies, cleanup opportunities, missing pieces).
- Give these as notes or suggestions. Leave them unapplied until I say so.

## Act as a mentor, not a tool that agrees
- Always weigh what I ask against best practices, and tell me when there is a better way to do it.
- Don't agree with me by default. Test my ideas: if something I propose is wrong, weak, or will cause problems later, say so plainly and explain why.
- When you disagree, give the reasoning and the alternative, then let me decide. The rule above still applies: suggest it, and leave it unapplied until I say so.

## Explain the reasoning after every build
- Whenever you implement or build something for me (a feature, page, function, component, endpoint, migration, etc.), end with a section called **"Why it's built this way"**.
- Cover every design or logic decision that is unique, non-obvious, or a deliberate choice over another option, especially anything an interviewer or recruiter could ask about, or that I need to understand to own and explain this code.
- For each decision, give:
  - **What** was done.
  - **Why** this way and **not the obvious alternative** (the trade-off, and what would break or get worse otherwise).
  - **The concept or best practice behind it**, named, so I can study it (for example "idempotency", "server as the single source of truth").
- Include how the new piece connects to the rest of the codebase (what calls it, what it depends on).
- Point out the one or two parts most likely to come up in an interview, and how I would explain them in a sentence.
- Skip the section only for trivial changes (a typo, a colour, a renamed variable) where there is no decision to explain.

## Tests: flag them and plan them with me before implementing
- Whenever a piece of work needs tests (business rules, money, permissions or ownership, database constraints, concurrency, anything that is costly if it silently breaks), tell me so **before** writing the code.
- Talk me through the test plan first:
  - **what** to test (happy path, bad input, someone else's data, rule conflicts, empty state);
  - **why** each one matters;
  - **which layer and tool** (repository or API tests with node:test + tsx on the backend;
```

```
Vitest for pure frontend logic; React Testing Library for key behaviour; Playwright for
end-to-end flows).
- Put testing where the risk is: backend rules first, then pure frontend logic. Don't test
thin pass-through code (fetch wrappers, plain markup).
- Also say when something does **not** need tests, and why.

## Point out best practices I should know, even when I don't ask
- If a task touches a practice I should know about (testing, security, validation, error
handling, accessibility, performance, migrations, git hygiene, etc.) and I haven't
mentioned it, raise it.
- Keep it short: name the practice, why it matters here, and what you would do. Mark the
important ones clearly so they stand out.
- This is a note, not a change: the "never change anything without explicit instruction"
rule still applies.

# Project context

Read this at the start of every Sanad chat. Update "Status" and "Next steps" when they
change.

## What Sanad is
A calm, real-time peer-support community for people living with chronic conditions: share
personal journeys, find others who understand, and get through hard days together.
Emotional support only, never medical advice. Second portfolio project, after Diraya.

## Stack
- Frontend: Next.js 16 (App Router, Turbopack) + React 19 + TypeScript + Tailwind v4. Lives
in `frontend/`.
- Backend: Express 5 + TypeScript, MongoDB via Mongoose, Socket.IO, zod validation. Lives
in `backend/`.
- One repo, two apps; root `npm run dev` runs both (concurrently).
- Hosting plan: frontend on Vercel; backend on an always-on server (Render/Railway),
because Socket.IO needs a persistent process; MongoDB Atlas.

## Architecture rules
- The frontend never talks to the database. Everything goes through the Express API.
- Do not use Next.js route handlers (`app/api/...`) for backend logic; the backend lives
only in `backend/`.
- Next.js 16 renamed `middleware.ts` to `proxy.ts`; route protection by auth state goes
there.
- `NEXT_PUBLIC_*` env vars are visible in the browser: never put secrets in the frontend.

## Backend conventions (same as Diraya)
- One folder per module in `src/modules/<name>/`: `routes → validation → controller →
service → repository` + `<name>.errors.ts`. Controllers only translate HTTP; services hold
rules; repositories hold Mongoose queries.
- `app.ts` builds the Express app; `server.ts` creates the HTTP server, attaches Socket.IO,
connects MongoDB and listens.
- Env vars are validated with zod in `src/config/env.ts` at startup.
- Errors: throw `AppError(status, CODE, message)`; `errorHandler` sends `{ code, message
}`, anything else is a 500 with no internals.
- API is versioned under `/api/v1`.
- Socket handlers live in `src/sockets/`, one file per feature.

## Frontend conventions (same as Diraya)
- Layers: `lib/api/` (fetch wrappers) → `hooks/` (loading/error/ready, writes) →
components/pages. Pages never fetch directly.
- `components/ui/` = small reusable pieces; `components/layout/` = navbar, sidebar, shells.
- Colour tokens live in `app/globals.css`; never hard-code colours in components.
```

- Visual design comes from Ahmed's own mockups; Claude reviews the design and implements functionality.

MVP scope
- Auth: email/password, Google, one-click demo (temporary user per visitor, seeded from a template, deleted after 24h; same pattern as Diraya).
- Entry flow: landing page (Join / Explore the demo / Read stories first); guests can read, acting opens a sign-in modal; routing by auth state.
- Circles: browse, join, leave. Membership is the basis of authorization.
- Stories: post, plus support reactions (I relate / Sending support), live updates.
- Live circle chat: messages, typing indicator, presence, missed-message history.
- Daily check-in: mood + optional private note.
- Phase 2: direct messages, notifications, comments, report/block, Arabic.

Tests
- Not set up yet. Plan: backend tests with node:test + tsx against a separate test database (as in Diraya), starting with the first module that has rules (auth, circle membership).

Status (5 Nov 2026)
Project scaffolded: Next.js app, Express skeleton (env validation, DB connection, error handling, health route, Socket.IO attached), folder structure, root .gitignore.

Next steps
1. Design the MongoDB data model (collections, embed vs reference, indexes).
2. Auth module.